

Digital Signal Processing Laboratory (EC39201) IIT Kharagpur

Experiment 1: Sampling

Name: Sanjay R, Rishant Mallick
Roll No.: 23EC10101, 23EC10067
Group No: 3
Date of submission: 21-08-2025



Aim

In this experiment, we perform:

- Sampling of a Sinusoidal Waveform
- Sampling at below Nyquist rate and effect of aliasing
- Spectrum of a Square Wave
- Interpolation or Upsampling
- Downsampling
- 2 Dimensional DFT

Theory

Most real-world signals are analog in nature. Computers, however, have finite memory and processing power. To process these signals, we must sample and quantize them into digital form.

Sampling

Sampling records analog information by periodic snapshots. If the rate is fast enough, the eye perceives continuity, as in motion pictures. It is the core of digitizing analog signals such as sound and images. Sampling is the process of converting a continuous-time signal $x(t)$ into a discrete-time signal $x[n]$ by taking measurements of the signal's amplitude at equally spaced time intervals.

Why Sampling?

Digital systems (computers, DSP processors, etc.) cannot directly process analog signals. Sampling allows analog signals (audio, images, biomedical, etc.) to be converted into a digital form.

Steps in Sampling Process

1. **Analog signal:** A continuous function of time.
2. **Sampling:** Take values at
$$T_s, 2T_s, 3T_s, \dots$$
3. **Discrete signal:** Sequence of numbers representing the sampled values.
4. (Later $\rightarrow$ quantization + encoding $\rightarrow$ full digitization).

Types of Sampling

- **Uniform Sampling** → samples at fixed intervals (most common).
- **Non-uniform Sampling** → samples at varying intervals (used in special DSP tasks).
- **Oversampling** → sample at rate much higher than Nyquist (reduces noise, easier filtering).
- **Undersampling / Bandpass Sampling** → sample below Nyquist under certain conditions (used in RF and communication systems).

Sampling Theorem

If a continuous-time signal contains no frequency component higher than B Hz, then uniform sampling at rate $f_s \geq 2B$ is sufficient for exact reconstruction:

$$x[n] = x_c(nT), \quad T = \frac{1}{f_s}$$

If $f_s < 2B$, overlapping spectra cause aliasing, making reconstruction impossible.

Nyquist-Shannon Sampling Theorem

To perfectly reconstruct a bandlimited signal (maximum frequency $f_{\max}$), the sampling frequency must satisfy:

$$f_s \geq 2f_{\max}$$

$\frac{f_s}{2}$ is called the **Nyquist rate**.

If $f_s < 2f_{\max}$, aliasing occurs (different frequency components overlap in sampled data).

Aliasing

When a signal is undersampled, higher frequency components fold back into the lower frequency range, distorting the signal.

Example: If $f_{\max} = 4$ kHz but $f_s = 6$ kHz, then $f_s < 2f_{\max}$, leading to aliasing.

Anti-aliasing filters (low-pass filters) are used before sampling to remove high-frequency components.

Reconstruction

The original continuous-time signal can be reconstructed from its samples using an **ideal low-pass filter** (sinc interpolation).

The interpolation formula is:

$$x(t) = \sum_{n=-\infty}^{\infty} x[n] \operatorname{sinc}\left(\frac{t - nT_s}{T_s}\right)$$

Part A: Sampling of a Sinusoidal Waveform

Theory

A sine wave is a smooth, continuous-time signal defined by its frequency, amplitude, and phase. When we sample a sine wave, we take discrete measurements of its amplitude at equal time intervals determined by the sampling frequency f_s .

Suppose we have a continuous-time sine wave:

$$x(t) = A \sin(2\pi ft)$$

where

- A = amplitude
- f = frequency of the sine wave

When we sample it with sampling frequency

$$f_s = \frac{1}{T_s},$$

we take samples at times $t = nT_s$:

$$x[n] = A \sin(2\pi f n T_s) = A \sin\left(2\pi \frac{f}{f_s} n\right)$$

So, the discrete-time sine depends on the **normalized frequency**:

$$f_d = \frac{f}{f_s}$$

Proper Sampling (No Aliasing)

If $f_s > 2f$, the samples uniquely represent the sine wave. If the sampling frequency is much higher than the sine wave frequency, the samples capture the shape of the sine wave accurately, and the original signal can be reconstructed.

Critical Sampling (Nyquist Rate)

If $f_s = 2f$, you just get two points per cycle, but the signal can still be reconstructed. At the Nyquist rate, only two points per cycle are captured, but reconstruction is still theoretically possible.

Undersampling (Aliasing)

If $f_s < 2f$, the sampled sequence looks like a lower-frequency sine wave (alias). When the sampling frequency is less than twice the sine wave frequency, aliasing occurs: the sampled points appear to form a sine wave of a different, lower frequency.

Thus, for a sine wave, correct sampling ensures that the original waveform can be fully represented in digital form without distortion.

Sampling of a Sine Wave

We consider:

$$x(t) = \sin(2\pi \cdot 1000t) + 2 \sin(2\pi \cdot 2000t) + 3 \sin(2\pi \cdot 4000t)$$

sampled at $f_s = 12$ kHz. The DFT is:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

Code

```
F = 12000;
N = 256;
t_axis = linspace(0,(N-1)/F,N);
original_axis = linspace(0,(N-1)/F,1000000);

w_1 = 2 * pi*1000;
w_2 = 2 * pi * 2000;
w_3 = 2 * pi * 4000;

x_t = sin(w_1*t_axis)+2*sin(w_2 * t_axis) + 3 *sin(w_3*t_axis);

x_fft = fftshift(fft(x_t));

subplot(1,2,1);
freq_axis = F/N * (-N/2:N/2-1);
figure(1);
plot(t_axis,x_t,'o');
hold on
plot(original_axis,sin(w_1*original_axis)+2*sin(w_2 * original_axis) + 3 *sin(w_3*original_axis));
ylabel("Amplitude");
xlabel("Time");
title("Original Signal with Samples");
hold off

subplot(1,2,2);
[peaks ,locations] = findpeaks(abs(x_fft));
plot(freq_axis,abs(x_fft),"LineWidth",3);
title("Complex Magnitude of FFT Spectrum");
xlabel("f (hz)");
ylabel("|fft(X)|");
text(freq_axis(locations),peaks,"FFT Peaks");
```

Figure 1: Code of Part A, Nyquist-Shannon Sampling Theorem and sampling above the Nyquist Frequency

Plots

Plots for $f_s = 12kHz$

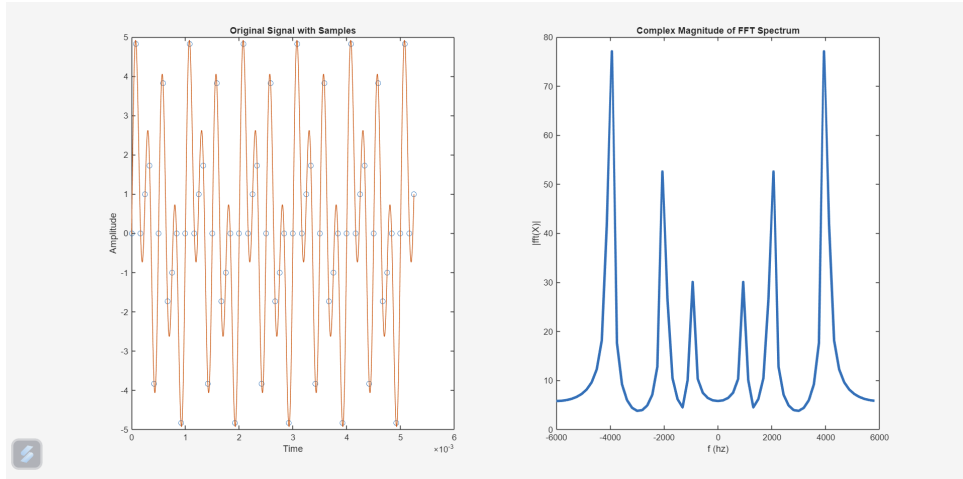


Figure 2: Number of points $N = 64$

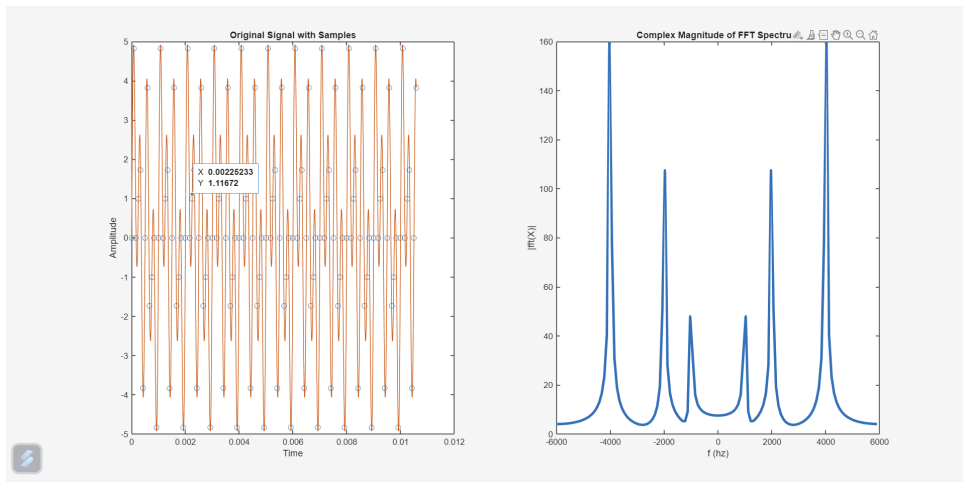


Figure 3: Number of points $N = 128$

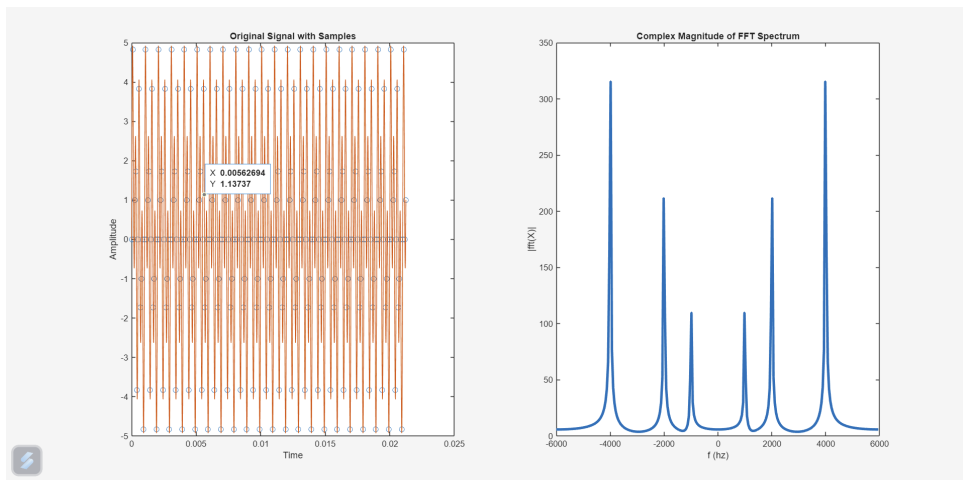


Figure 4: Number of points $N = 256$

Part B: Sampling below Nyquist rate and Aliasing

Similar analysis with $f_s = 8, 5, 4$ kHz. Aliasing occurs when $f_s < 2f_{\max}$.

Theory

According to the Nyquist-Shannon sampling theorem, a bandlimited signal with maximum frequency $f_{\max}$ can be perfectly reconstructed if the sampling frequency satisfies:

$$f_s \geq 2f_{\max}$$

If the sampling frequency is chosen such that $f_s < 2f_{\max}$, then the Nyquist criterion is violated. In this case, different frequency components of the signal overlap in the sampled spectrum, which leads to distortion. This effect is known as **aliasing**.

In aliasing, high-frequency components of the signal appear as lower frequencies in the sampled version, creating an incorrect representation of the original signal. Once aliasing occurs, it cannot be removed by signal processing. To prevent this, an **anti-aliasing filter** (low-pass filter) is typically applied before sampling to limit the signal bandwidth.

Plots

For $f_s = 8\text{kHz}$

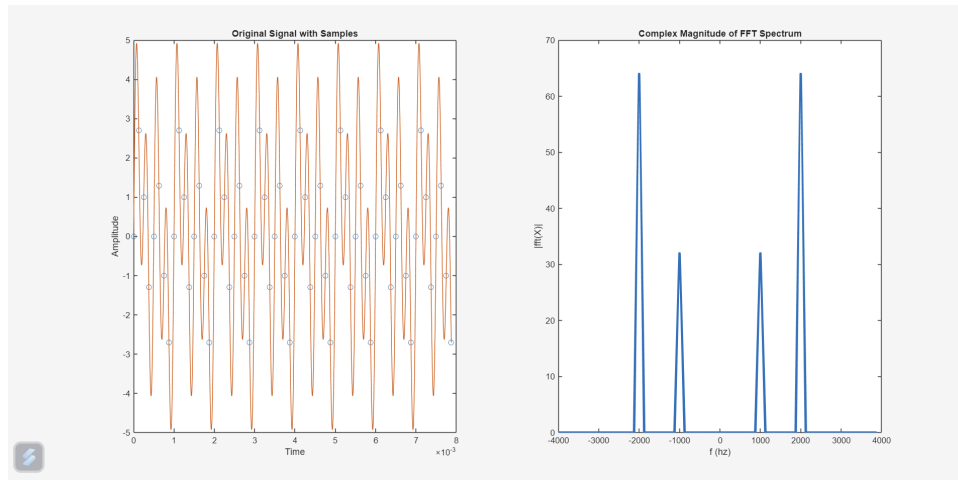


Figure 5: Number of points $N = 64$

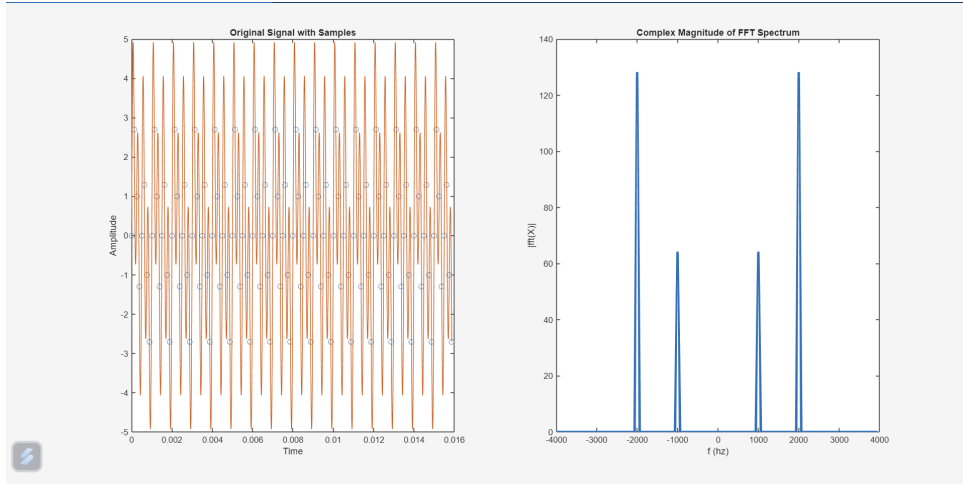


Figure 6: Number of points $N = 128$

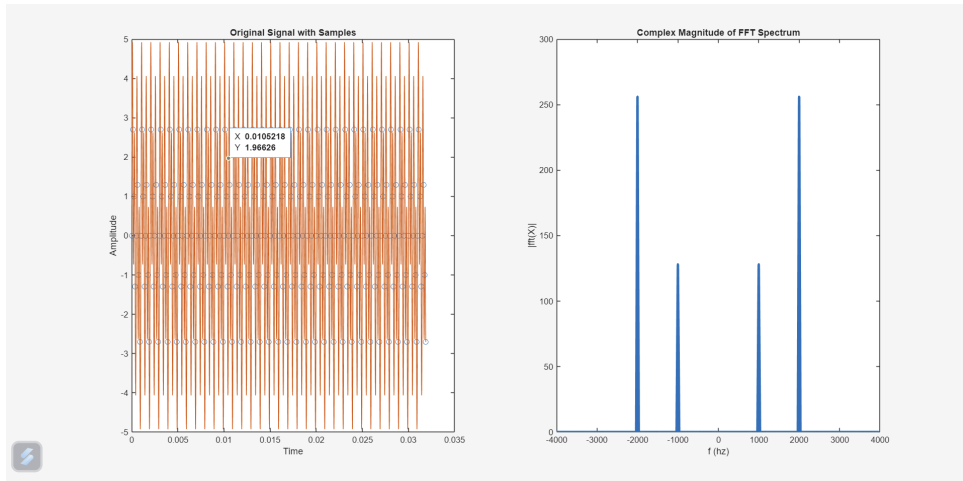


Figure 7: Number of points $N = 256$

For $f_s = 5kHz$

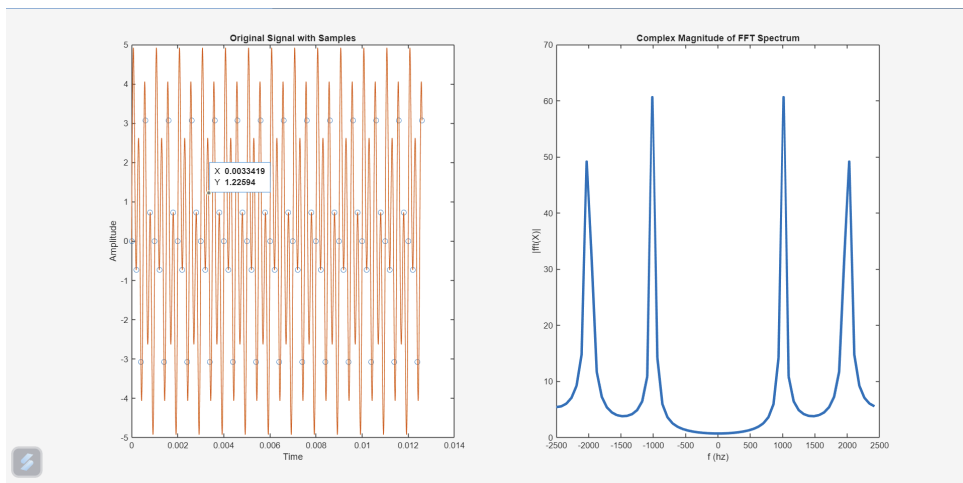


Figure 8: Number of points $N = 64$

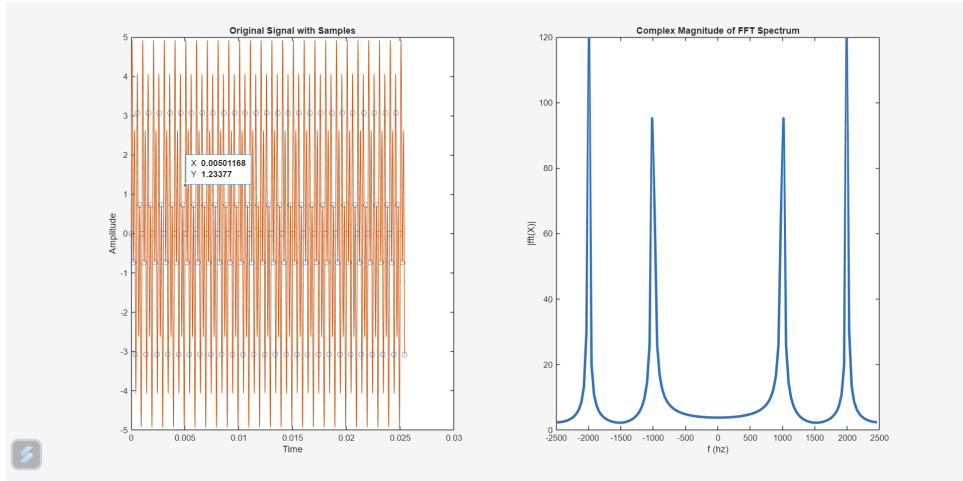


Figure 9: Number of points $N = 128$

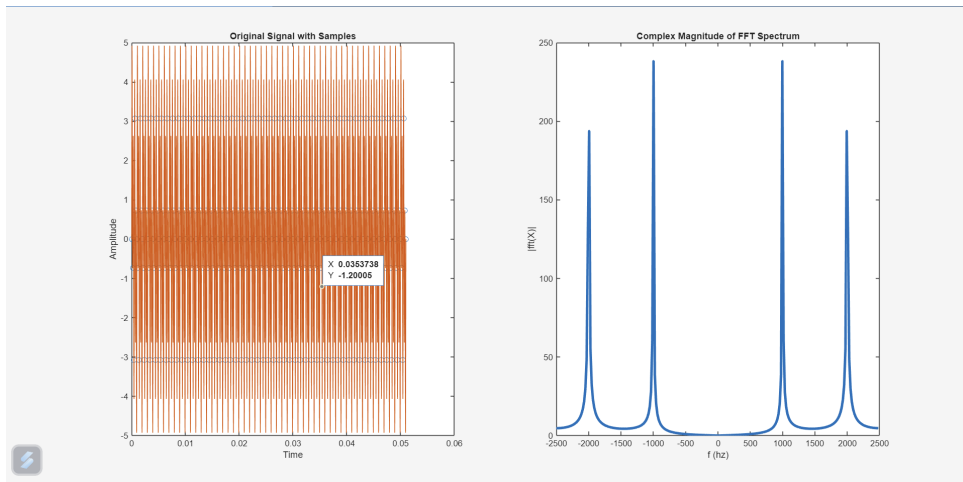


Figure 10: Number of points $N = 256$

For $f_s = 4kHz$

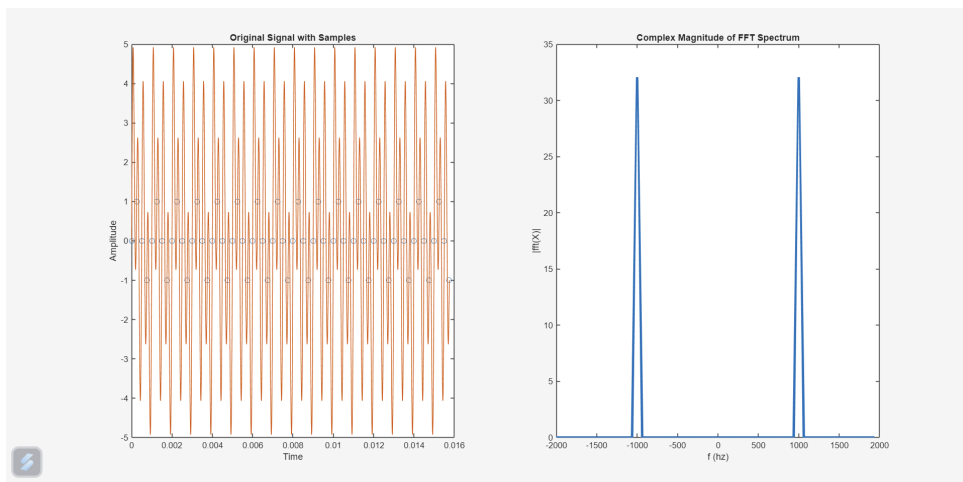


Figure 11: Number of points $N = 64$

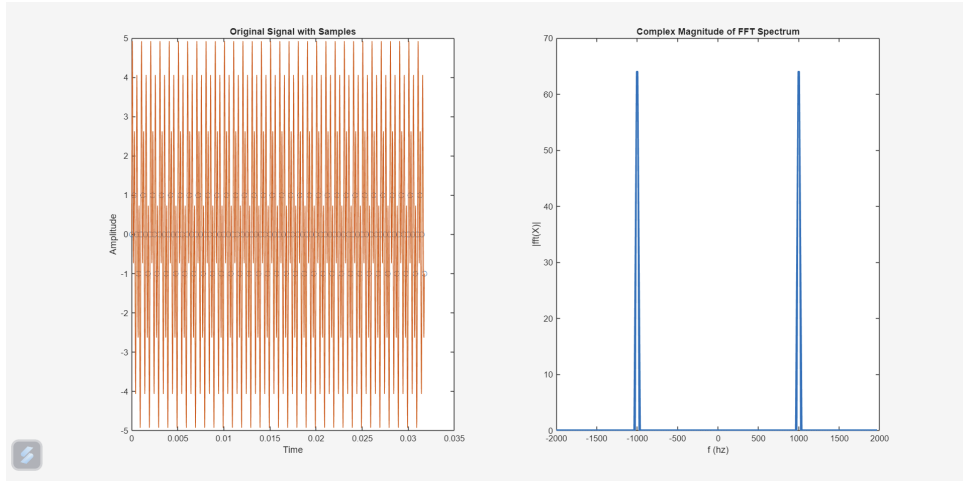


Figure 12: Number of points $N = 128$

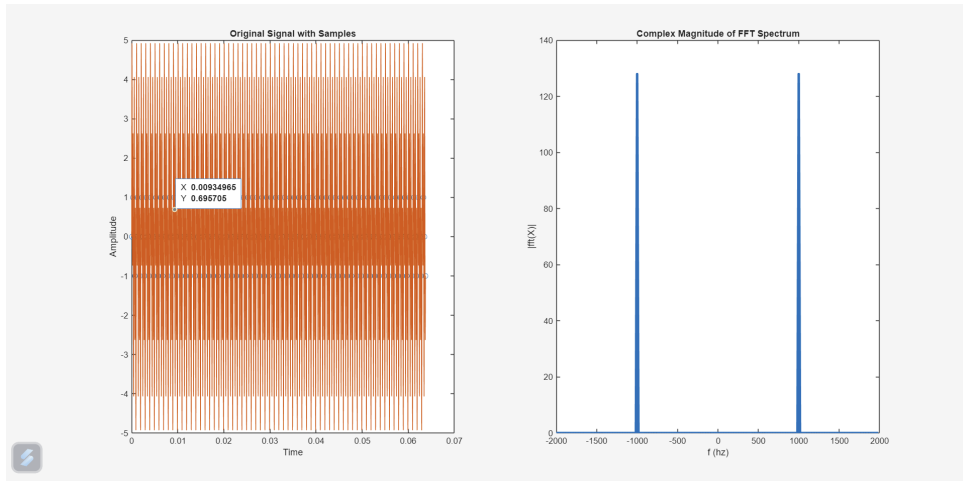


Figure 13: Number of points $N = 256$

Part C: Spectrum of a Square Wave

A square wave (1 kHz) sampled at 20kHz , $N = 256$. Since a Square wave is an infinite sum of sinusoids, we see multiple peaks in the Fourier Transform Plot.

Theory

A square wave is a periodic signal that alternates between two amplitude levels. Being periodic, it can be expressed as a Fourier series. A symmetric square wave of amplitude A , period T , and fundamental frequency $f_0 = \frac{1}{T}$ can be written as:

$$x(t) = \frac{4A}{\pi} \left(\sin(2\pi f_0 t) + \frac{1}{3} \sin(2\pi(3f_0)t) + \frac{1}{5} \sin(2\pi(5f_0)t) + \dots \right)$$

Properties of the Spectrum

- Contains only **odd harmonics** of the fundamental frequency.

- Amplitude of the n -th harmonic is

$$\frac{4A}{n\pi}, \quad n = 1, 3, 5, \dots$$

- No even harmonics are present.
- Phase alternates between 0 and π depending on the harmonic.

Thus, the spectrum of a square wave consists of impulses at odd multiples of f_0 with amplitudes inversely proportional to the harmonic number.

Code

```
time_period = 1e-3;
sampling_frequency = 20e3;
number_of_points = 256;

time_axis = linspace(0,(number_of_points-1)/sampling_frequency,number_of_points);
sine_wave = sin(2*pi*time_axis/time_period);

square_wave = sign(sine_wave);

subplot(1,2,1);
plot(time_axis,square_wave);
ylim([-1.5 1.5]);
xlabel("Time Axis");
ylabel("Amplitude");
title("Signal Plot");

square_wave_fft = fftshift(fft(square_wave));
frequency_axis = (-number_of_points/2:number_of_points/2-1)*sampling_frequency/number_of_points;

subplot(1,2,2);
plot(frequency_axis,abs(square_wave_fft),"LineWidth",3);
k = abs(square_wave_fft);
xlabel("Frequency");
ylabel("Magnitude of FFT");
title("FFT Plot");
peak_locations = abs(square_wave_fft)>19;
text(frequency_axis(peak_locations),k(peak_locations),"Peaks");
```

Figure 14: Code for Generating a Square wave and plotting its Fourier Transform, note the peaks are also already labelled

Plots

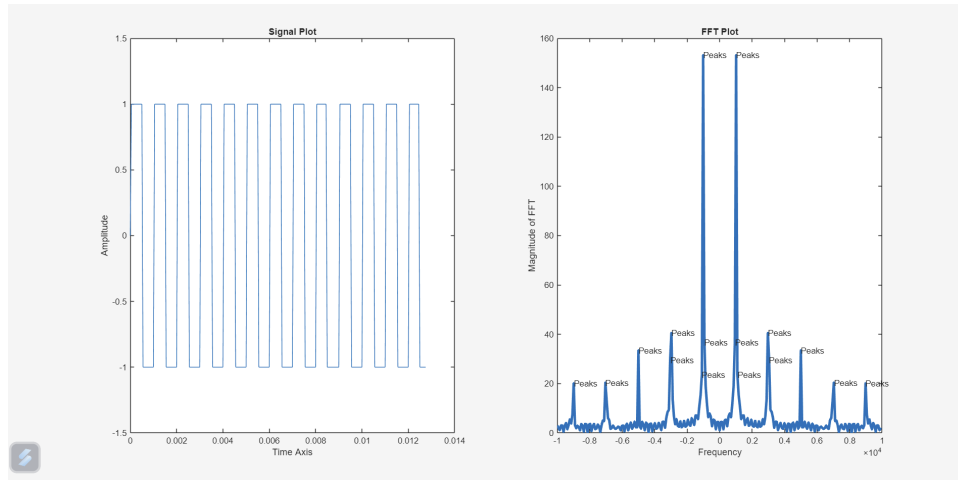


Figure 15: Plot of the square wave and its Fourier Transform

Part D: Interpolation or Upsampling

Interpolation / Upsampling

Interpolation (or upsampling) is the process of **increasing the sampling rate** of a discrete-time signal. Upsampling introduces zeros between samples. This artificially increases the sampling rate, filtering then smoothens out the signal. If a signal $x[n]$ is upsampled by a factor of L , the operation can be described as:

$$x_L[n] = \begin{cases} x[\frac{n}{L}], & n \text{ is a multiple of } L \\ 0, & \text{otherwise} \end{cases}$$

This means that $L - 1$ zeros are inserted between every original sample.

Frequency-Domain Effect

In the frequency domain, upsampling by L compresses the spectrum by a factor of L and produces $L - 1$ spectral replicas. To eliminate these unwanted images, the signal is passed through a **low-pass interpolation filter**.

Key Points

- Upsampling increases the sampling frequency from f_s to Lf_s .
- Zero-insertion creates spectral images (copies of the spectrum).
- An interpolation filter is required to reconstruct the smooth higher-rate signal.

For this part, we took $N = 512$ and the signal $x(t) = \sin(2\pi \cdot 1000t) + 2 \sin(2\pi \cdot 2000t)$

Code

```
frequency_1 = 1e3;
frequency_2 = 2e3;
no_of_points = 512;
sample_frequency = 12e3;

time_axis= linspace(0,(no_of_points-1)/sample_frequency,no_of_points);
signal = sin(2*pi*frequency_1*time_axis) + 2*cos(2*pi*frequency_2*time_axis);
%subplot(1,3,1);
figure(1);
plot(time_axis,signal);
xlim([0,0.01]);
xlabel("Time Axis");
ylabel("Amplitude");
title("Original Signal Sampled at 12kHz");

signal_upsampled = upsample(signal,2);
filtered_signal = lowpass(signal_upsampled,sample_frequency/2,sample_frequency*2);
time_axis_new = linspace(0,(no_of_points-1)/sample_frequency,no_of_points*2);
%subplot(1,3,2);
figure(2);
plot(time_axis_new,filtered_signal);
xlim([0,0.01]);
xlabel("Time Axis");
ylabel("Amplitude of the filtered signal");
title("Filtered Signal plot sampled at 24kHz");
```

Figure 16: Part 1 of the Upsampling Code

```
og_signal = sin(2*pi*frequency_1*time_axis_new) + 2*cos(2*pi*frequency_2*time_axis_new);
%subplot(1,3,3);
figure(3);
plot(time_axis_new,og_signal);
xlim([0,0.01]);
xlabel("Time Axis");
ylabel("Amplitude");
title("Original Signal Sampled at 24kHz");

figure(4);
subplot(1,2,1);
frequency_axis = linspace(-pi,pi,no_of_points);
plot(frequency_axis,abs(fftshift(fft(signal))));
xlabel("Frequency Axis");
ylabel("Magnitude of FFT");
title("Original Signal FFT");

subplot(1,2,2);
frequency_axis = linspace(-pi,pi,2*no_of_points);
plot(frequency_axis,abs(fftshift(fft(signal_upsampled))));
xlabel("Frequency Axis");
ylabel("Magnitude of FFT");
title("Upsampled FFT");
```

Figure 17: Part 2 of the Upsampling Code

Plots

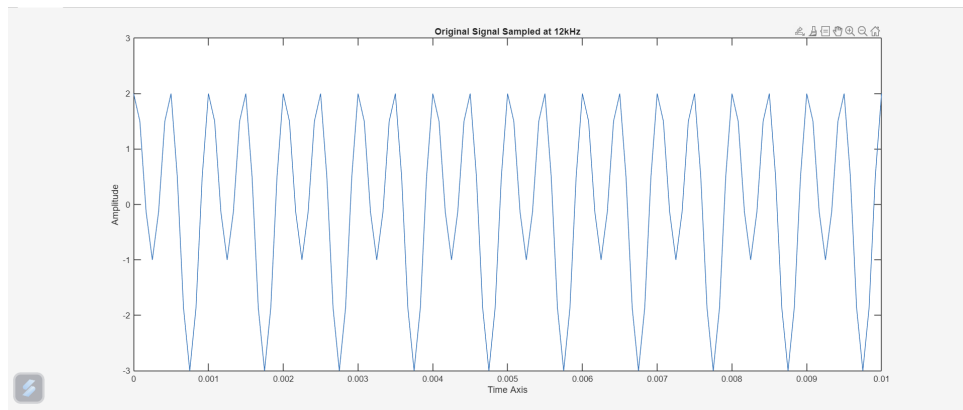


Figure 18: Original Signal Sampled at $12kHz$

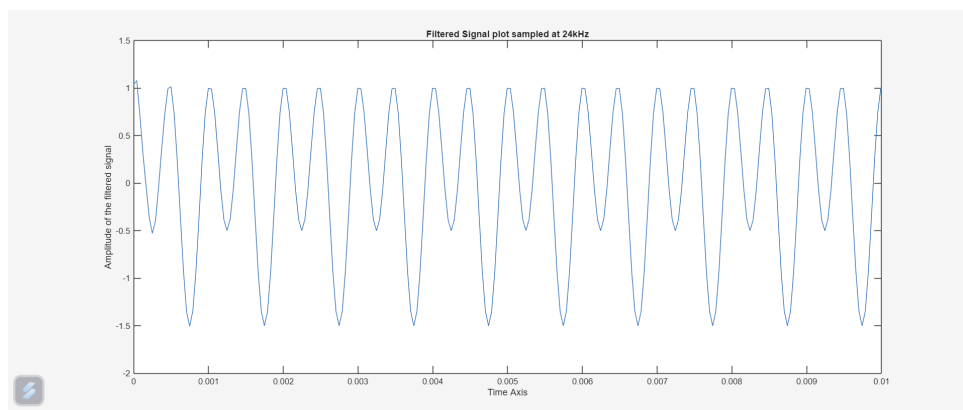


Figure 19: Filtered Signal Sampled at $24kHz$

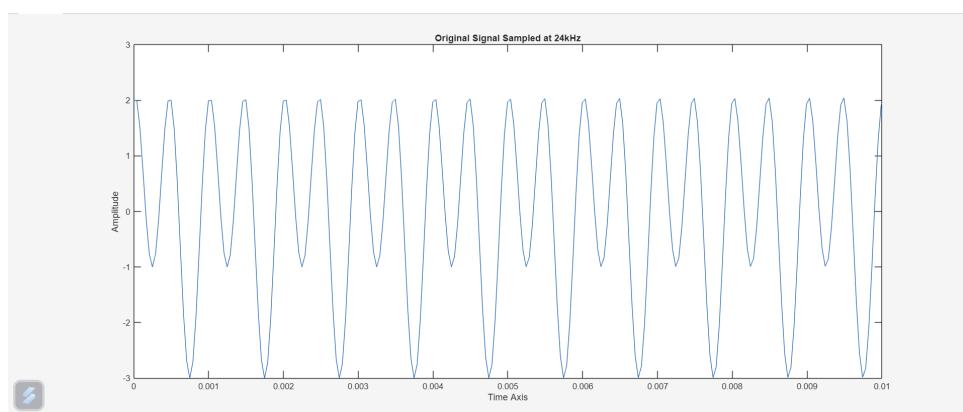


Figure 20: Original Signal Sampled at $24kHz$

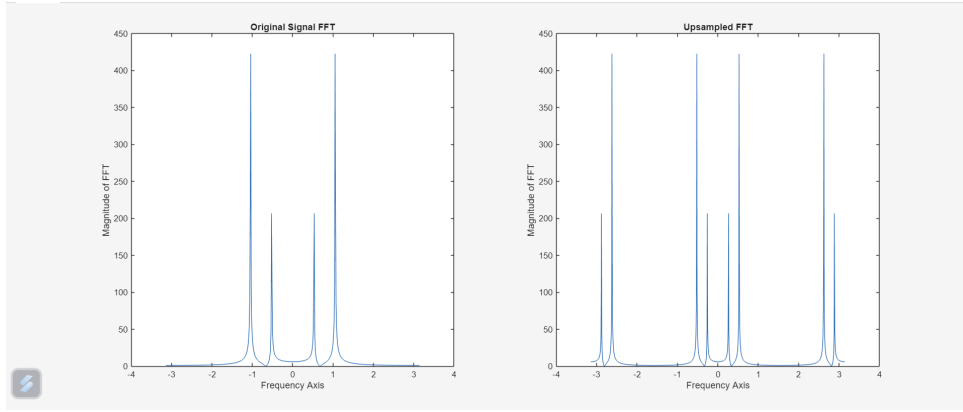


Figure 21: Original Signal Spectra vs the Upsampled Signal Spectra, notice how the axis for the upsampled signal is shrunk in half

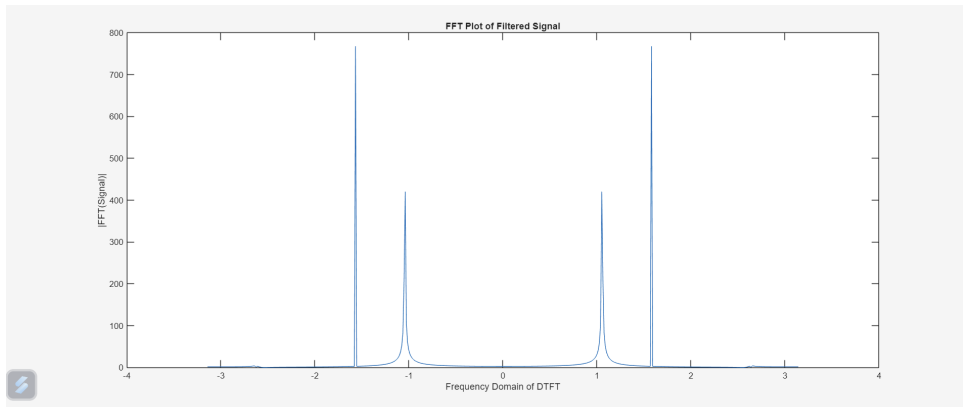


Figure 22: Filtered Signal Spectrum

Part E: Downsampling

Downsampling / Decimation

Downsampling (or decimation) is the process of **reducing the sampling rate** of a discrete-time signal. In contrast to upsampling, we remove every alternate point if we have too much information; if the downsampling is too much, aliasing might occur. If a signal $x[n]$ is downsampled by a factor of M , the operation is defined as:

$$x_d[n] = x[nM]$$

This means that only every M -th sample of the original sequence is kept, and the rest are discarded.

Frequency-Domain Effect

In the frequency domain, downsampling by M expands the spectrum by a factor of M . If the original signal is not bandlimited to $\frac{f_s}{2M}$, spectral overlap occurs, resulting in **aliasing**. To avoid this, an **anti-aliasing low-pass filter** is applied before downsampling.

Key Points

- Downsampling reduces the sampling frequency from f_s to $\frac{f_s}{M}$.
- Without filtering, aliasing can occur due to overlapping frequency components.
- Used in multirate DSP, audio compression, and communication systems.

In this case to show downsampling we took the signal $x(t) = \cos(2\pi \cdot 2000t) + 2 \sin(3\pi \cdot 2000t)$ and to demonstrate aliasing, we took the signal $x(t) = \cos(2\pi \cdot 2000t) + 2 \sin(3\pi \cdot 2000t) + 3 \cos(2\pi \cdot 8000t)$

Code

```
f1 = 2e3;
f2 = 3e3;
f3 = 8e3;

sampling_frequency = 24e3;
number_of_points = 1024;

time_axis = linspace(0,(number_of_points-1)/sampling_frequency,number_of_points);

input_signal = 2*cos(2*pi*f1*time_axis) + 3*cos(2*pi*f2*time_axis);
figure(1);
plot(time_axis,input_signal);

hold on;

down_sampled_signal = downsample(input_signal,2);

filtered_signal = lowpass(down_sampled_signal,5e3,sampling_frequency/2);

new_time_axis = linspace(0,(number_of_points-1)/sampling_frequency,number_of_points/2);
plot(new_time_axis,filtered_signal);
xlabel("Time Axis");
ylabel("Amplitude");
title("Discrete Time Plot of the Signal");
xlim([0,0.01]);
legend("Original Signal","Filtered Signal");
```

Figure 23: Part 1 of the Downsampling Code


```

figure(2);
input_frequency_axis = linspace(-pi,pi,number_of_points);
filtered_frequency_axis = linspace(-pi,pi,number_of_points/2);
plot(input_frequency_axis,abs(fftshift(fft(input_signal))));
hold on;
plot(filtered_frequency_axis,abs(fftshift(fft(down_sampled_signal))));

title("FFT Plots");
xlabel("Frequency Domain of DTFT");
ylabel("|FFT(Signal)|");
grid on;
hold off;
legend("Input Signal", "Down Sampled Signal");

figure(3);
plot(filtered_frequency_axis,abs(fftshift(fft(filtered_signal))));
title("FFT Plot of Filtered Signal")
xlabel("Frequency Domain of DTFT");
ylabel("|FFT(Signal)|");

% To show the effect of Aliasing;

original_signal = 2*cos(2*pi*f1*time_axis) + 3*cos(2*pi*f2*time_axis)+4*sin(2*pi*f3*time_axis);
aliased_signal = downsample(original_signal,2);

figure(4);

```

Figure 24: Part 2 of the Downsampling Code

```

original_signal = 2*cos(2*pi*f1*time_axis) + 3*cos(2*pi*f2*time_axis)+4*sin(2*pi*f3*time_axis);
aliased_signal = downsample(original_signal,2);

figure(4);
plot(input_frequency_axis*sampling_frequency/(2*pi),abs(fftshift(fft(original_signal))));
title("Observing Aliasing ,Input Signal");
xlabel("Frequency Domain of DTFT");
ylabel("|FFT(Signal)|");
legend("Original Signal FFT");

hold on;
rectangle("Position", [-12000,0,24000,2000]);
hold off;

figure(5);
plot(filtered_frequency_axis*sampling_frequency/(4*pi),abs(fftshift(fft(aliased_signal))));
title("Observing Aliasing ,Aliased Signal");
xlabel("Frequency Domain of DTFT");
ylabel("|FFT(Signal)|");
xlim([-7000,7000]);
hold on;
rectangle("Position", [-6000,0,12000,1000]);
hold off;
legend("Aliased Signal FFT");

```

Figure 25: Part 3 of the Downsampling Code

Plots

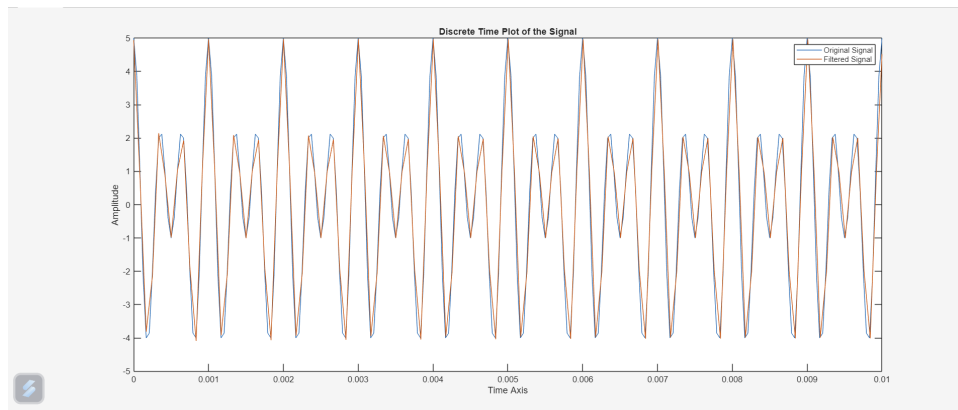


Figure 26: The Original Signal, overlaid with filtered signal

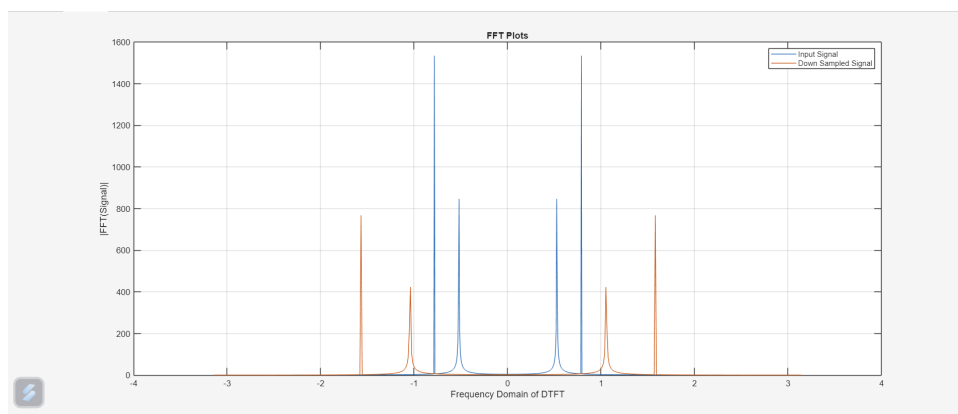


Figure 27: FFT Plot of the Original Signal Overlaid with the filtered signal

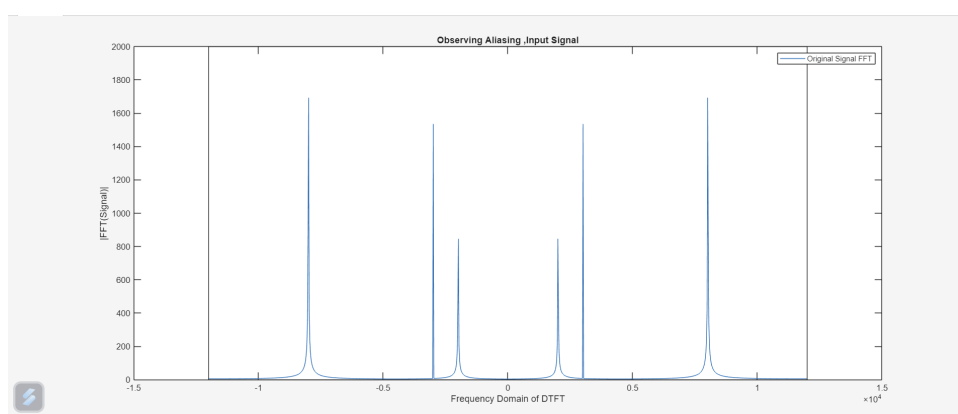


Figure 28: Original Signal before downsampling, note that the black box is the ideal filter to be applied before reconstructing the original signal

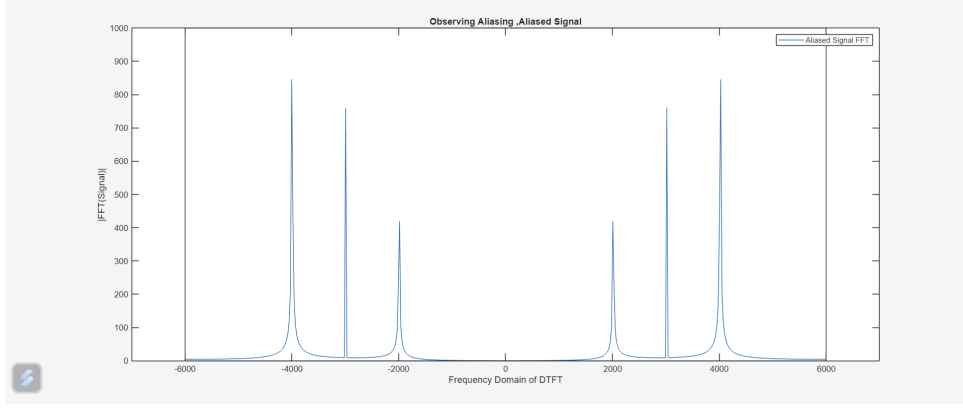


Figure 29: Signal getting Aliased after downsampling, note that the black box is the ideal filter to be applied before reconstructing the original signal

Part E: 2 Dimensional DFT

Two-Dimensional Discrete Fourier Transform (2D DFT)

The 2D Discrete Fourier Transform is an extension of the 1D DFT to two dimensions, widely used in image and multidimensional signal processing.

For a 2D sequence $x[m, n]$ of size $M \times N$, the 2D-DFT is defined as:

$$X[k, l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x[m, n] e^{-j2\pi(\frac{km}{M} + \frac{ln}{N})}, \quad 0 \leq k < M, \quad 0 \leq l < N$$

The inverse 2D-DFT is given by:

$$x[m, n] = \frac{1}{MN} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} X[k, l] e^{j2\pi(\frac{km}{M} + \frac{ln}{N})}$$

Properties

- **Separability:** The 2D DFT can be computed by applying 1D DFT along rows, then columns.
- **Periodicity:** Input and output are periodic in both dimensions.
- **Symmetry:** For real-valued signals, the 2D DFT exhibits conjugate symmetry.
- **Applications:** Image compression, filtering, denoising, and feature extraction.

Code

```
image = imread("cameraman.jpg");

fft_image = fft2(image);

fk = fspecial('disk',5); % a flat circular filter
filtered_image = imfilter(image,fk,'replicate');
downsampled_filtered_image = filtered_image(1:2:end,1:2:end);
figure(1);
imshow(downsampled_filtered_image);
output_1 = mat2gray(log(1+abs(fftshift(fft2(downsampled_filtered_image))))));
figure(2);
imshow(output_1);

down_sampled_image = image(1:2:end,1:2:end);
output_2 = mat2gray(log(1+abs(fftshift(fft2(down_sampled_image))))));
figure(3)
imshow(down_sampled_image);
figure(4);
imshow(output_2);
```

Figure 30: Code for the 2D FFT

Plots



Figure 31: Image after just down sampling

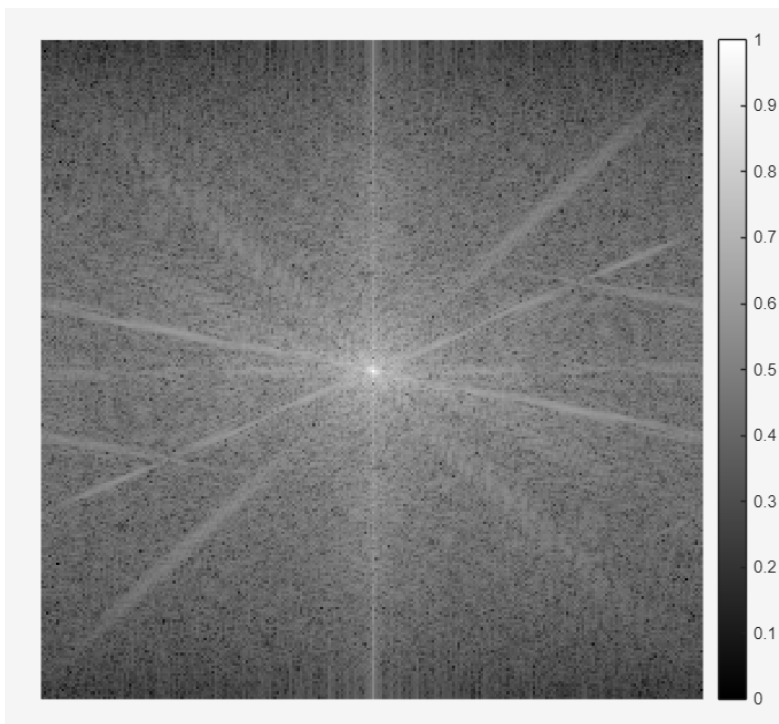


Figure 32: $\text{Log}(1 + |FFT|)$ Plot of image after downsampling



Figure 33: Image after filtering and downsampling

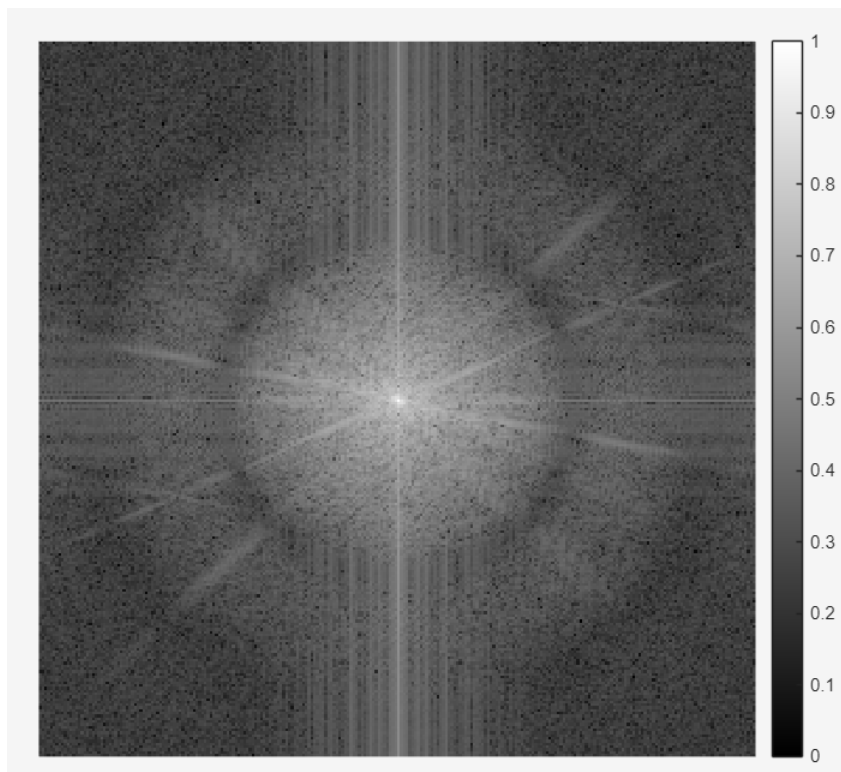


Figure 34: $\text{Log}(1 + |FFT|)$ Plot of Image after Downsampling and Filtering

Discussion

In this experiment, we studied the basics of digital signal processing in MATLAB. First we learnt about the Nyquist-Shannon Sampling Theorem, and how we can perfectly reconstruct the original signal if the signal is bandlimited upto f_c and we can perfectly reconstruct the signal if the sampling frequency $f_s > 2f_c$. If this condition is not met, we get aliasing effect, where the higher frequency components get shifted into the passband. This can be explained using a simple rotating wheel. If the wheel spins fast enough, the wheel starts rotating in the other direction. As our eyes cannot sample at a very high frequency, Aliasing is observed.

A square wave, represented in terms of Fourier series, contains infinite sum of Cosinoids. So in the frequency domain, we get ideal delta function spikes at odd harmonics. Since our points and time are both limited, we observe spectral leakage as shown in the FFT Plot.

Upsampling is a method of artificially increasing the rate of sampling by inserting a zero between each of the sample. This has the effect of shrinking the FFT Plot by half in the digital frequency axis. This follows from the properties of Discrete Time Fourier Transform. Applications of upsampling is image processing, digital audio processing to improve signal processing.

In contrast, downsampling reduces the number of points in time domain. This has the effect of expanding the digital frequency axis. Advantages of Downsampling include better efficiency, lowering storage needs and to simplify analysis.

Image processing is the method of transforming images into a more useful format by applying algorithms to enhance quality or extract specific information. This can involve improving visual aspects like color and contrast or separating an image into distinct segments for detailed analysis. It's a broad field with both analog and digital forms, though digital image processing, which uses computer algorithms on digital images, is the most common. In this experiment, we learnt the basics of 2D FFT and Downsampling of an image.

A 2D FFT (Two-Dimensional Fast Fourier Transform) transforms a 2D signal (like an image) from the spatial domain to the frequency domain. This allows for analyzing the frequency components present in the signal. Conversely, a 2D Inverse Fast Fourier Transform (IFFT) can reconstruct the original signal from its frequency representation. Downsampling of an image introduces high frequency components and causes more "peaks" in the 2D Plot.